

When does breaking a problem into smaller tasks help?

A recursive language model (RLM) can use code and ask another model for help. Our experiments tested different parts of that idea.

We supplied some splits ourselves. Our code asked a separate helper to check each candidate against the requirements.

We also trained the model to choose smaller questions. It proposed a plan for answering a question using several documents.

In crafting, we trained the model to choose its next action. It looked up recipes and made the needed parts.

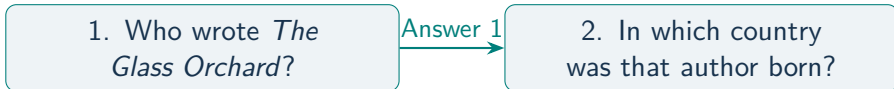
These are different abilities. We have not yet demonstrated a generally useful, learned tree of recursive helpers.

Can the model learn which smaller questions to ask?

MuSiQue contains questions that need evidence from several documents.

An example of the original question

In which country was the author of *The Glass Orchard* born?



We taught the model example question lists, then rewarded plans that led to correct answers.

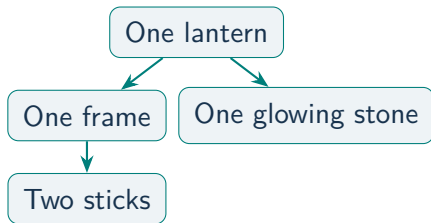
Finding: This system did not reliably beat answering directly, and used more text.

Invented example. The model wrote the question list; code ran it. This was not a learned recursive tree.

A crafting game makes the outcome easy to check.

In TextCraft-Synth, success means making the requested number of new items. The game checks the inventory increase when the model finishes.

Goal: make one lantern.



Arrows mean “needs.”

Simplified illustration. Real tests use unfamiliar names and longer chains. Scores count whole-task success, with no partial credit.

Start with two sticks and one glowing stone.

1. Look up the lantern and frame recipes.
2. Make the frame from the sticks.
3. Combine the frame and stone into a lantern.

Show the relevant fact before the action.

Supervised fine-tuning (SFT) teaches a next action from an example's goal and observations.

Original input

Goal: make a lantern.

History: no frame mentioned.



Next action to learn

Look up the frame recipe.

Repaired input

Goal: make a lantern.

Lookup: a lantern needs a frame.



The same action to learn

Look up the frame recipe.

We rebuild the earlier lookup history. **Each example keeps the same action to learn.**

Simplified illustration. The repair uses a known solution; input histories, their lengths and computing work change.

The repair helped after retraining and on more goals.

Each bar counts successful crafting attempts out of 32. Both models learned the same actions for the same number of training updates.



Each row tests eight goals in two recipe worlds, twice each. There are **16 distinct goals in total**; the first two rows reuse the same goals.

Selected TextCraft-Synth panels, not the full benchmark; Qwen3-4B. More training helps the original examples too. A teacher that discovers recipes in order also performs well.

Let code write the known ingredient list.

After a recipe lookup, code can write the ingredients for the **model's chosen item and quantity**.

Example: for “make one lantern,” code writes “one frame and one glowing stone.”

Model writes the full action.

42%
of attempts succeeded.

Code writes the ingredients.

50%
of attempts succeeded.

We changed the interface, not the model's weights. Code supplies no materials.

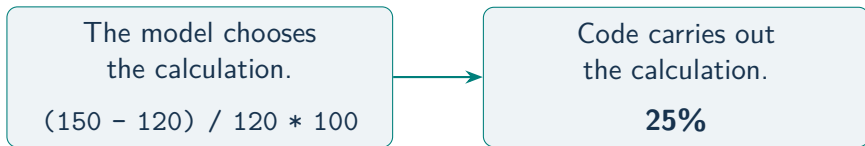
Selected TextCraft-Synth panels: 323/768 versus 381/768 whole-task successes. There are 48 distinct goals, four recipe worlds and repeated seeds, not 768 independent problems.

We also tested letting code do the arithmetic.

Across five datasets, splitting documents among helpers did not consistently improve answers.

A simplified financial question

Revenue rose from 120 to 150. What was the percentage increase?



Of 508 FinQA questions, **51 answers matched the recorded number without code, versus 108 with code.** Each used one model call.

Qwen3-4B; previously examined FinQA development questions. Rounding and reference errors limit the score; not official accuracy. Invented example; no extra training.

What did we learn from reward training?

Reinforcement learning (RL) rewards attempts that actually finish the task. We started with an SFT-trained model and trained on eight crafting goals.



Every point tests the **same eight other goals twice each**. The extra success after two updates did not persist.

Lesson: A gain must survive new examples and comparison with simply continuing SFT.

Selected TextCraft-Synth training-split goals; recipes overlap. No checkpoint was picked for its score here. Three small updates are not a verdict on RL.

The broader tests changed how we interpret progress.

A programmed split helped select candidates that met stated requirements. Helpers checked one candidate at a time. The model did not learn to choose that split.

News-classification gains weakened on fresh examples. The AG News tests showed why repeating training is not enough: we also need new articles.

Conversation-retrieval RL mostly improved exact copying. In MRCR, the model reproduced earlier replies more accurately, but did not find the right reply more often.

These are separate studies. We now ask which ability changed, whether it helps the whole task, and whether it carries over to new problems.

Can the teaching repair survive stronger tests?

Research question

Can rebuilding what the learner sees make existing worked examples more useful, without changing the actions we teach?

Test new problems. Use new recipe structures, not just more attempts at familiar goals.

Use stronger comparisons. Include a teacher that discovers recipes in order, and give the original examples more training.

Find the limits. Try another model and test whether the gain is mainly easier copying of item names.

A paper candidate, not an established general method. What evidence would make it useful beyond this game?